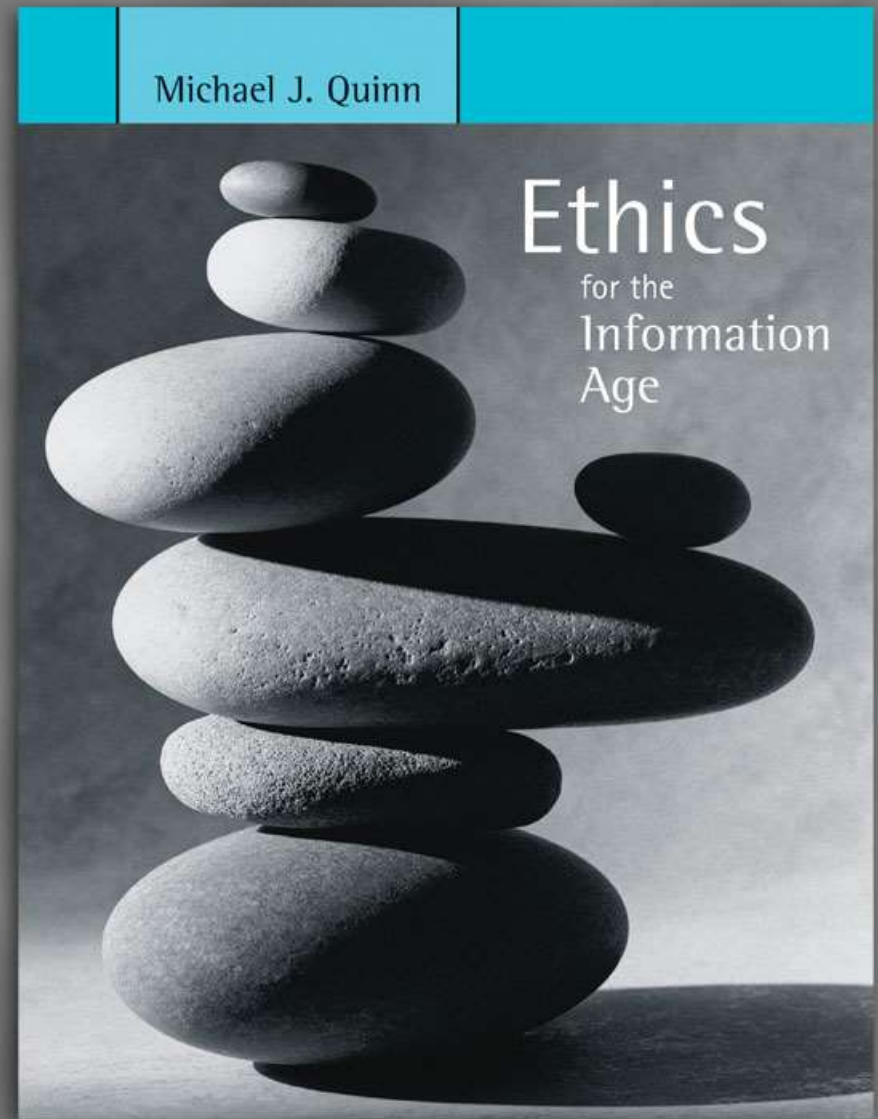


Chapter 7

Computer Reliability

Small and BIG ERRORS





Errors

- **Playing a game - program crashes**
- **Incorrect bill in the mail**
- **Software bug in an investor firm**
- **You get a phone bill for \$50,000...**
- **You are trapped into your rental car in Dallas in August because the car computer fails, locks all doors and turns off air conditioner...**



Failure reasons vary widely...

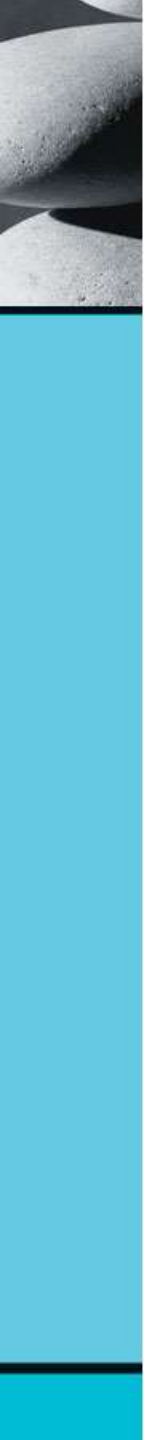
- **Failures may be due to**
 - data entry,
 - poor design,
 - hardware malfunction
 - **Software malfunction...**
- **Big or small errors can result in big or small failures**
 - Telephone system outages - three faulty computer instructions hidden in software changes that were sent out without sufficient testing
 - Company thought it was too small to change

Some BIG errors

- **A one-character typographical error in one line of a program[CACM Jan 1990 p.4-7]**
 - The lost satellite incident
 - Safeguards?
- **A timing-related problem in the Patriot antimissile system control software**
 - Gulf war (Lee: the day the phones stopped, Primus books 1992)
- **Error in computer program**
 - used to calculate the doses in irradiation therapy that caused patients to receive 35% less radiation than prescribed (1981-1991, N. Staffordshire Hospital, UK)

Living with Computer System Unreliability

- A computer is just a component of a larger system
 - What does this mean about the whole system?
- What do we mean when we say that this is a *well-engineered system*?
 - *That it can tolerate the malfunction of any single component without failure*
- Computer simulations
- Large software engineering projects
- Software warranties
 - How much responsibility should sw manufacturers take??
- What is the Uniform Computer Information Act (UCITA)?



Chapter Overview

- Introduction
- Data-entry or data-retrieval errors
- Software and billing errors
- Notable software system failures
- Therac-25
- Computer simulations
- Software engineering
- Software warranties

Introduction

- Computer systems are sometimes unreliable
 - Erroneous information in databases
 - Misinterpretation of database information
 - Malfunction of **embedded systems**
 - Embedded systems are everywhere!
- Effects of computer errors
 - Inconvenience
 - Bad business decisions
 - Fatalities

Data-Entry or Data-Retrieval Errors

- Wrong data entered
- Incorrect interpretation of the data
- Example: Nov 2000 elections in Florida...
 - Computer error Disfranchised voters as being felons when they were charged with misdemeanors
- False arrests
 - NCIC 40 million records of crimes
 - Sheila Jackson instead of Shirley Jackson spent one night in jail...
 - Two Roberto Hernandez men, left tatoo, same ht, wt, brown hair, same bday, jailed wrong one twice for days...
- Who is responsible for the Accuracy of NCIC records
 - No more need for FBI to check accuracy (Privacy Act of 1974)
 - DOJ says it is not practical
 - Data comes in from many many sources and gets fused



Disfranchised Voters

- November 2000 general election
- Florida disqualified thousands of voters
- Reason: People identified as felons
- Cause: Incorrect records in voter database
- Consequence: May have affected election's outcome

False Arrests due to NCIC errors

- Sheila Jackson Stossier mistaken for Shirley Jackson
 - Arrested and spent five days in detention
- Roberto Hernandez mistaken for another Roberto Hernandez
 - Arrested twice and spent 12 days in jail
- Terry Dean Rogan arrested after someone stole his identity
 - Arrested five times, three times at gun point

Accuracy of NCIC Records

- March 2003: Justice Dept. announces FBI not responsible for accuracy of NCIC information
- Exempts NCIC from some provisions of Privacy Act of 1974
- Should government take responsibility for data correctness?



Dept. of Justice Position

- Impractical for FBI to be responsible for data's accuracy
- Much information provided by other law enforcement and intelligence agencies
- Agents should be able to use discretion
- If provisions of Privacy Act strictly followed, much less information would be in NCIC
- Result: fewer arrests of criminals

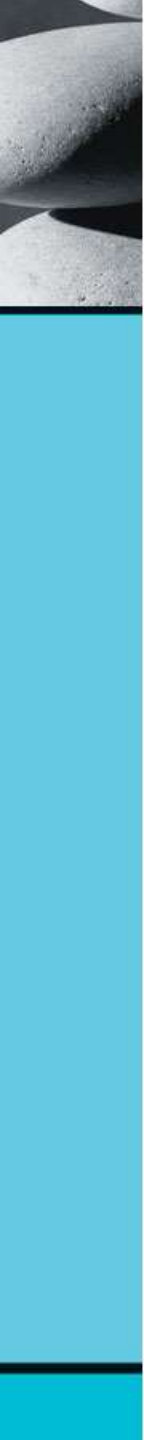


Position of Privacy Advocates

- Accuracy of NCIC records more important than ever
- Number of records is increasing
- As More erroneous records are entered into the database → more false arrests are made

Analysis of NCIC case: Database of Stolen Vehicles

- > 1 million cars stolen every year in the US
 - Owners suffer emotional, financial harm
 - Raises insurance rates for all
- Before: Transporting stolen car across a state line
 - Before we had NCIC, greatly reduced chance of recovery
 - After NCIC, nationwide stolen car retrieval
- At least 50,000 recoveries annually due to NCIC
- Few stories of faulty information causing false arrests
 - Benefit > harm only small number of people due to false arrests (utilitarian principle)
 - → Creating database the right action



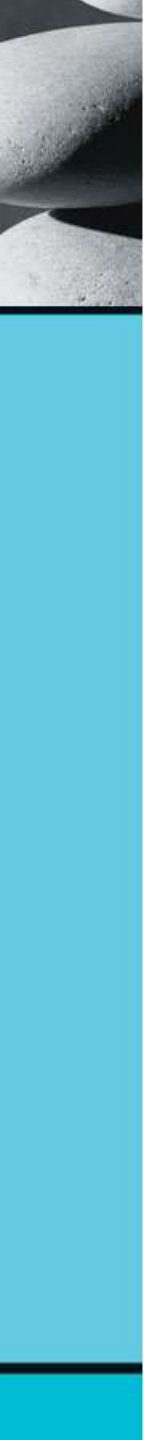
Software and Billing Errors

- Errors leading to system malfunctions
- Errors leading to system failures
- Analysis: E-retailer posts wrong price, refuses to deliver



Errors Leading to System Malfunctions

- Qwest sends incorrect bills to cell phone customers
- Faulty USDA beef price reports
- U.S. Postal Service returns mail addressed to Patent and Trademark Office
- Spelling and grammar error checkers increased errors
- BMW on-board computer failure



Errors Leading to System Failures

- Los Angeles County – USC Medical Center laboratory computer
- Japan's air traffic control system
- Chicago Board of Trade
- London International Financial Futures and Options Exchange



Analysis: E-Retailer Posts Wrong Price, Refuses to Deliver

- Amazon.com in Britain offered iPaq for 7 pounds instead of 275 pounds
- Orders flooded in
- Amazon.com shut down site, refused to deliver unless customers paid true price
- Was Amazon.com wrong to refuse to fill the orders?



Rule Utilitarian Analysis

- Imagine rule: A company must always honor the advertised price
- Consequences
 - More time spent proofreading advertisements
 - Companies would take out insurance policies
 - Higher costs → higher prices
 - All consumers would pay higher prices
 - Few customers would benefit from errors
- Conclusion
 - Rule has more harms than benefits
 - Amazon.com did the right thing

Kantian Analysis

- Buyers knew 97.5% markdown was an error
- They attempted to take advantage of Amazon.com's stockholders
- They were not acting in "good faith"
- Buyers did something wrong



Safety critical systems

- Systems with a component of real-time control that can have a direct life-threatening impact
- Examples:
 - Aircraft, air traffic control
 - Nuclear reactor control
 - Missile systems
 - Medical treatment systems
- Safety critical software used in the design of physical systems can have massive impact
- Bridges, building designs, selection of waste disposal sites



Failures often due to a combination of factors

- Nancy Leveson
 - An investigation of Therac 25 accidents analysis
 - Most accidents involving complex technology are caused by a combination of organizational, managerial, technical and sociological or political factors
 - Preventing accidents requires paying attention to ALL the root causes.



Managing Murphy's Law Report

IEEE spectrum vol 26, no.6, pp 24-27, June 1989

- How to engineer a minimal risk system?
- How can hardware faults and human lapses be dealt with in systems so complex that not all potential fault paths may be foreseen?
- Engineers must face questions such as:
 - What can go wrong?
 - How likely is it to happen?
 - What range of consequences might there be?
 - How could they be averted or mitigated?
 - How much RISK should be tolerated or accepted during normal operation?
 - How can risk be measured, reduced and managed?



Account for human element

- What could human operators do to the system for better or worse?
- Can their responses be predicted?
- How safe is safe enough if human life is at stake?
- Formal risk analysis= attempts to pin down and quantify the answers to these questions
- Point: Build-into the design ways to mitigate, not after product is built and operated.

What is risk analysis?

- **A combination of the PROBABILITY of an undesirable event with the MAGNITUDE of each and every foreseeable consequence (loss of x)**
- **Consequences range in magnitude from “no, never mind”, to catastrophic**
- **Reliability: related to risk but not the whole operation**
- **Reliability is the probability of a component of a system performing its mission of a certain length of time but this does not account for external factors**



Risk v. Hazard

- Hazard; the potential for injury or danger
 - Ex. Toxic materials or radioactive fuel that a chemical or nuclear plant must manipulate
- Risk: Identify and Quantify failure probability
 - Risk adds to the likelihood of the injury or danger occurring
- Hazard analysis: is just one step in Risk Analysis

Assessing and Quantifying Risk

- A complete risk analysis of any system **requires a “cradle to grave” systems approach to safety** that foresees, for example, waste disposal and end-user’s risk right from the start, in the DESIGN PHASE
 - NOT AS AN ADD-ON (a patch)
- Qualitative assessment before quantitative assessment
 - For each of the system’s phases or functionalities, list the associated risk sensitivities
 - For each phase, the system’s operations should be diagrammed and the logical relations to the other parts determined.
- Design objective must be on par with:
 - Operability, security, privacy, maintainability, risk analysis.



Most useful technique

1. **Identify failure nodes and effects analysis**= identify all the ways a piece of equipment might fail
 2. **Event tree analysis** -=identify potential accidents with forward thinking
 3. **Fault tree analysis**=deduction of causes of failure with reverse thinking
- These three complement each other
 - Understanding the impact of interaction of the parts is very important



Risk probability

- Can vary, depending on the phase, the status of the critical component or piece of equipment, the test data used.
- Destructive outside forces: another important factor
 - Natural hazards, earthquakes, terrorists, other...
 - These kinds of events may cause unrelated system elements to fail simultaneously
 - Depends on system history: important to collect data on previous behavior that system can analyze



Accept and manage risk

- Make social judgment call
 - cost/benefit ratio analysis
 - Know cost to Surrounding environment and Human life
- How does this relate to a particular workplace?
 - Some workplaces have a higher tolerance of risk
- Alternatives must be in place in case of failure
 - Need redundancy
 - Need safety shut-off switch
 - Containment plans
 - Shielding
 - Escape routes.
- Use estimates from other similar systems



Friendly interface to monitor, interact with system, train

- Leave little room for error
- Help operator understand a complex system
- Proper training
- Frequent testing and drilling
- Alleviate poor operational design
- Physiological factors - fatigue
- Take into account human behavior: how humans respond to crisis



Replace HW controllers with SW ones

- Embedded systems - most are real time systems= process data from sensors as events occur (cell phones, air bags)
- Sw plays key role in system functionality
- Why are hw controllers replaced with sw?
 - Faster, cheaper, more data, less energy, do not wear out, but less reliable



Notable Software System Failures

- Patriot Missile
- Ariane 5
- AT&T long-distance network
- Robot missions to Mars
- Denver International Airport



Patriot Missile

- Designed as anti-aircraft missile
- Used in 1991 Gulf War to intercept Scud missiles
- One battery failed to shoot at Scud that killed 28 soldiers
- Designed to operate only a few hours at a time
- Kept in operation > 100 hours
- Tiny truncation errors added up
- Clock error of 0.3433 seconds → tracking error of 687 meters



Ariane 5

- Satellite launch vehicle
- 40 seconds into maiden flight, rocket self-destructed
 - \$500 million of uninsured satellites lost
- Statement assigning floating-point value to integer raised exception
- Exception not caught and computer crashed
- Code reused from Ariane 4
 - Slower rocket
 - Smaller values being manipulated
 - Exception was impossible

AT&T Long-Distance Network

- Significant service disruption
 - About half of telephone-routing switches crashed
 - 70 million calls not put through
 - 60,000 people lost all service
 - AT&T lost revenue and credibility
- Cause
 - Single line of code in error-recovery procedure
 - Most switches running same software
 - Crashes propagated through switching network



Robot Missions to Mars

- Mars Climate Orbiter
 - Disintegrated in Martian atmosphere
 - Lockheed Martin design used English units
 - Jet Propulsion Lab design used metric units
- Mars Polar Lander
 - Crashed into Martian surface
 - Engines shut off too soon
 - False signal from landing gear



Denver International Airport

- BAE built automated baggage handling system
- Problems
 - Airport designed before automated system chosen
 - Timeline too short
 - System complexity exceeded development team's ability
- Results
 - Added conventional baggage system
 - 16-month delay in opening airport
 - Cost Denver \$1 million a day



Therac-25

- Genesis of the Therac-25
- Chronology of accidents and AECL responses
- Software errors
- Post mortem
- Moral responsibility of the Therac-25 team

Genesis of the Therac-25

- AECL and CGR built Therac-6 and Therac-20
- Therac-25 built by AECL
 - PDP-11 an integral part of system
 - Hardware safety features replaced with software
 - Reused code from Therac-6 and Therac-20
- First Therac-25 shipped in 1983
 - Patient in one room
 - Technician in adjoining room



Chronology of Accidents and AECL Responses

- Marietta, Georgia (June 1985)
- Hamilton, Ontario (July 1985)
- First AECL investigation (July-Sept. 1985)
- Yakima, Washington (December 1985)
- Tyler, Texas (March 1986)
- Second AECL investigation (March 1986)
- Tyler, Texas (April 1986)
- Yakima, Washington (January 1987)
- FDA declares Therac-25 defective (February 1987)



Software Errors

- Race condition: order in which two or more concurrent tasks access a shared variable can affect program's behavior
- Two race conditions in Therac-25 software
 - Command screen editing
 - Movement of electron beam gun



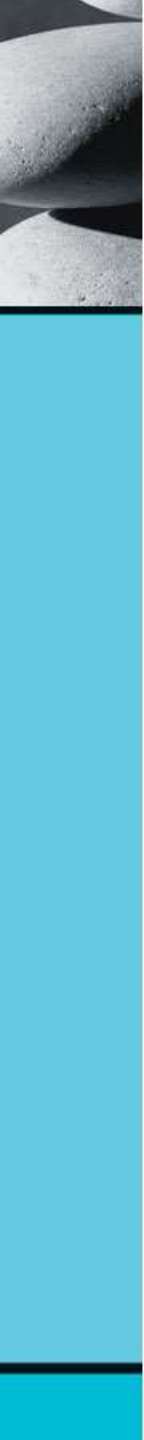
Post Mortem

- AECL focused on fixing individual bugs
- System not designed to be fail-safe
- No devices to report overdoses
- Software lessons
 - Difficult to debug programs with concurrent tasks
 - Design must be as simple as possible
 - Documentation crucial
 - Code reuse does not always lead to higher quality
- AECL did not communicate fully with customers



Moral Responsibility of the Therac-25 Team

- Conditions for moral responsibility
 - Causal condition: actions (or inactions) caused the harm
 - Mental condition
 - Actions (or inactions) intended or willed -OR-
 - Moral agent is careless, reckless, or negligent
- Therac-25 team morally responsible
 - They constructed the device that caused the harm
 - They were negligent



Computer Simulations

- Uses of simulation
- Validating simulations

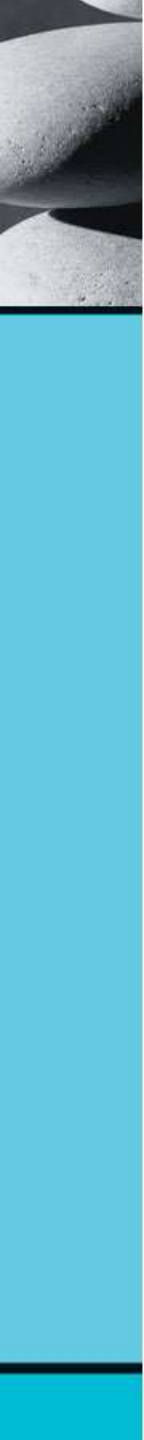


Uses of Simulations

- Simulations replace physical experiments
 - Experiment too expensive or time-consuming
 - Experiment unethical
 - Experiment impossible
- Model past events
- Understand world around us
- Predict the future

Validating Simulations

- Verification: Does program correctly implement model?
- Validation: Does the model accurately represent the real system?
- Validation methods
 - Make prediction, wait to see if it comes true
 - Predict the present from old data
 - Test credibility with experts and decision makers



Software Engineering

- Specification
- Development
- Validation (testing)
- Software quality is improving



Specification

- Determine system requirements
- Understand constraints
- Determine feasibility
- End products
 - High-level statement of requirements
 - Mock-up of user interface
 - Low-level requirements statement



Development

- Create high-level design
- Discover and resolve mistakes, omissions in specification
- CASE tools to support design process
- Object-oriented systems have advantages
- After detailed design, actual programs written
- Result: working software system



Validation (Testing)

- Ensure software satisfies specification
- Ensure software meets user's needs
- Challenges to testing software
 - Noncontinuous responses to changes in input
 - Exhaustive testing impossible
 - Testing reveals bugs, but cannot prove none exist
- Test modules, then subsystems, then system

Software Quality Is Improving

- Standish Group tracks IT projects
- Situation in 1994
 - One-third of projects cancelled before completion
 - One-half of projects had time and/or cost overruns
 - One-six of projects completed on time / on budget
- Situation in 2002
 - One-sixth of projects cancelled
 - One-half of projects had time and/or cost overruns
 - One-third of projects completed on time / on budget

Software Warranties

- Shrinkwrap warranties
- Are software warranties enforceable?
- Uniform Computer Information Transaction Act
- Moral responsibility of software manufacturers

Shrinkwrap Warranties

- Some say you accept software “as is”
- Some offer 90-day replacement or money-back guarantee
- None accept liability for harm caused by use of software

Are Software Warranties Enforceable?

- Article 2 of Uniform Commercial Code
- Magnuson-Moss Warranty Act
- *Step-Saver Data Systems v. Wyse Technology and The Software Link*
- *ProCD, Inc. v. Zeidenberg*
- *Mortensen v. Timberline Software*



Uniform Computer Information Transaction Act

- National Conference of Commissioners on Uniform State Laws drafted UCITA
- Under UCITA, software manufacturers can
 - License software
 - Prevent software transfer
 - Disclaim liability
 - Remote disable licensed software
 - Collect information about how software is used
- UCITA applies to software in computers, not embedded systems

Arguments in Favor of UCITA

- Article 2 of the UCC not appropriate for software
- UCITA recognizes there is no such thing as perfect software
- UCITA prevents software fraud

Arguments Against UCITA

- Customers should be allowed to purchase software
- UCITA bans giving away software
- UCITA removes software from protections of Magnuson-Moss Act
- UCITA codifies practice of hiding warranty
- UCITA allows “trap doors”
- UCITA restricts free speech
- Fuzzy line between embedded systems & computers
- UCITA is unlikely to pass without amendments



Moral Responsibility of Software Manufacturers

- If vendors were responsible for harmful consequences of defects
 - Companies would test software more
 - They would purchase liability insurance
 - Software would cost more
 - Start-ups would be affected more than big companies
 - Less innovation in software industry
 - Software would be more reliable
- Making vendors responsible for harmful consequences of defects may be wrong
- Consumers should not have to pay for bug fixes